Mess Update — Technical Report

Nov 7, 2026 · @Kenny Nauna

1. Project summary

Student	Kenny Nauna
Student number	24202171
Unit	IS325 Mobile App Development, PNG University of Technology
Repository	github.com/KennyNauna26/mess-update (private, tag `v1.0`)
Stack	React Native, Expo SDK 57 (managed), Expo Router, TypeScript, Supabase (Auth, Postgres, RLS)
Test account	Email: kennynauna66@gmail.com · Password: 42anekst2026 (student role, with sample feedback)

Mess Update lets students check the campus mess before walking there: today's menu and serving status, the latest queue level with an estimated wait range and its age, where the mess is relative to them, and a private log of their own meal ratings. Meal-time reminders are scheduled on the device.

Problem. Students walk to the mess without knowing what is being served or how long the queue is, and often arrive to a long line or a finished meal.

Users. Students use the app. The mess administrator maintains meals and queue reports in the Supabase dashboard; the database only lets accounts with the admin role write them.

Queue data is reported, not sensed. The app does not count people. Each queue report records who posted it and when, the app always shows how old the report is, and reports older than 60 minutes carry a "may be outdated" warning.

Final scope. Two navigation patterns (a five-tab bar and stack drill-downs), 12 screens, full feedback CRUD with ownership enforced by Row Level Security, Maps & Location (Group A) and local scheduled notifications (Group B), loading/empty/error states on every data source, and a signed APK and AAB built with EAS. Cut deliberately: in-app admin screens, meal photos, weekly menu, favourites and remote push notifications (see Section 7).

2. Requirements

All 18 must-have stories are built; 3 of 5 should-haves are built; the could-haves were cut to protect the must-haves.

ID	User story	Priority	Status
US01	As a student, I want to create an account so that I can access Mess Update.	Must	Built
US02	As a student, I want to log in so that I can use the app's features.	Must	Built
US03	As a student, I want to log out so that nobody else can use my account on this phone.	Must	Built
US04	As a student, I want the app to remember my login so that I do not sign in every time I open it.	Must	Built
US05	As a student, I want to reset a forgotten password with an emailed code so that I can regain access.	Must	Built
US06	As a student, I want to view today's breakfast, lunch and dinner so that I know what is served before going.	Must	Built
US07	As a student, I want to open a meal's details so that I can decide whether to go.	Must	Built
US08	As a student, I want to see the current queue status so that I can decide to go now or later.	Must	Built
US09	As a student, I want an estimated wait range so that I can plan when to visit.	Must	Built
US10	As a student, I want to see when the queue was last updated so that I know if it is recent.	Must	Built
US11	As a student, I want to see the mess on a map so that I can find it.	Must	Built
US12	As a student, I want to see my distance and walking time to the mess so that I know how long it takes.	Must	Built
US13	As a student, I want to rate a meal (1–5) with a comment so that I can record my experience.	Must	Built
US14	As a student, I want a list of my feedback and a detail view so that I can review it.	Must	Built

ID	User story	Priority	Status
US15	As a student, I want to edit my feedback so that I can correct it.	Must	Built
US16	As a student, I want to delete my feedback after confirming so that I can remove it safely.	Must	Built
US17	As a student, I want reminders before each meal's serving time so that I do not miss it.	Must	Built
US18	As a student, I want the last loaded menu and queue when offline, with a retry option, so that the app stays useful.	Must	Built
US19	As a student, I want to choose which meals I am reminded about so that I only get the ones I want.	Should	Built
US20	As a student, I want today's queue history so that I can spot quieter times.	Should	Built
US21	As a student, I want to report the queue I see so that others get fresher information.	Should	Cut
US22	As a student, I want walking directions in Google Maps so that I can get there.	Should	Built
US23	As a student, I want the mess address from reverse geocoding so that I can confirm the location.	Should	Cut
US24	As a student, I want a weekly menu so that I can plan ahead.	Could	Cut
US25	As a mess administrator, I want to upload a photo of each meal so that students see what is served.	Could	Cut

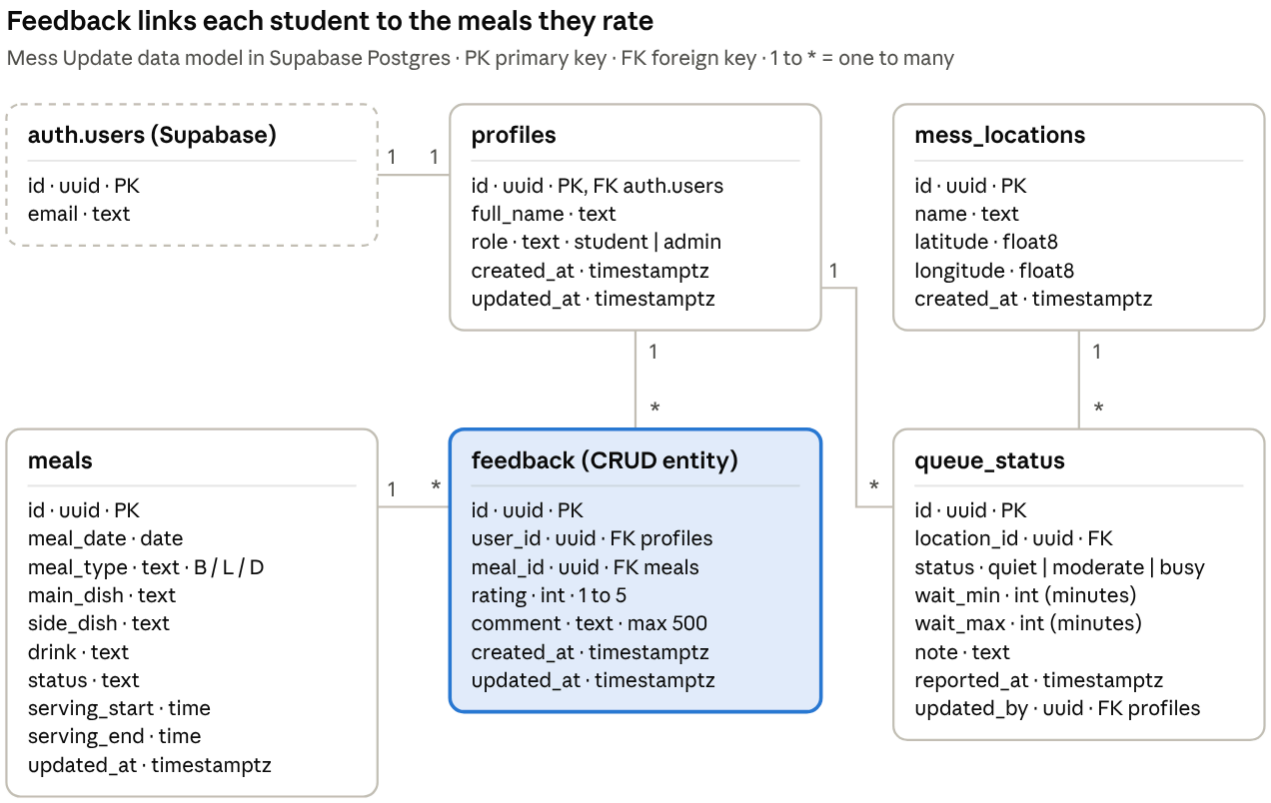
How the mandatory technical requirements are met

Brief	Requirement	How Mess Update meets it
3.1	Expo managed, Expo Router, TypeScript, ⩾ 6 screens, 2 navigation patterns	Expo SDK 57; file-based routes; strict TypeScript; 12 screens; bottom tabs + stack drill-down
3.2	Auth, protected routes, context, no password handling, reset	Supabase Auth SDK; `(app)` layout redirects signed-out users; `AuthProvider` context; email-code password reset and email confirmation

Brief	Requirement	How Mess Update meets it
3.3	Remote DB, ⩾ 2 related entities, full CRUD, DB-level ownership, local storage	Supabase Postgres, 5 tables; feedback CRUD (validated form, list + detail, pre-filled edit, confirm-before-delete); RLS on every table; AsyncStorage for the session, the offline cache and reminder preferences
3.4	⩾ 2 external APIs, one from Group A, four states each	Maps & Location (Group A) and local notifications (Group B), each with loading, success, empty and error states
3.5	Robustness and UX	Loading indicators, inline Zod messages, retry on errors, empty states, single theme file, safe areas and keyboard handling
3.6	Security	Only the publishable key ships; `.env` git-ignored with `.env.example` committed; no service-role key; RLS is the security boundary

3. Data model

Five Supabase Postgres tables. Feedback is the CRUD entity: one student has many feedback rows, and one meal receives many.



data model · 5 tables, 5 relationships

`auth.users` is managed by Supabase Auth (dashed); a trigger creates the matching `profiles` row on sign-up. Each queue report belongs to one mess location and records the admin who posted it.

Constraints: `unique (user_id, meal_id)` (one review per student per meal); `rating between 1 and 5`; comment ⩽ 500 characters; `unique (meal_date, meal_type)`; `serving_end > serving_start`; `wait_max ⩾ wait_min`. The current queue is the newest `queue_status` row by `reported_at`.

Row Level Security (enabled on every table)

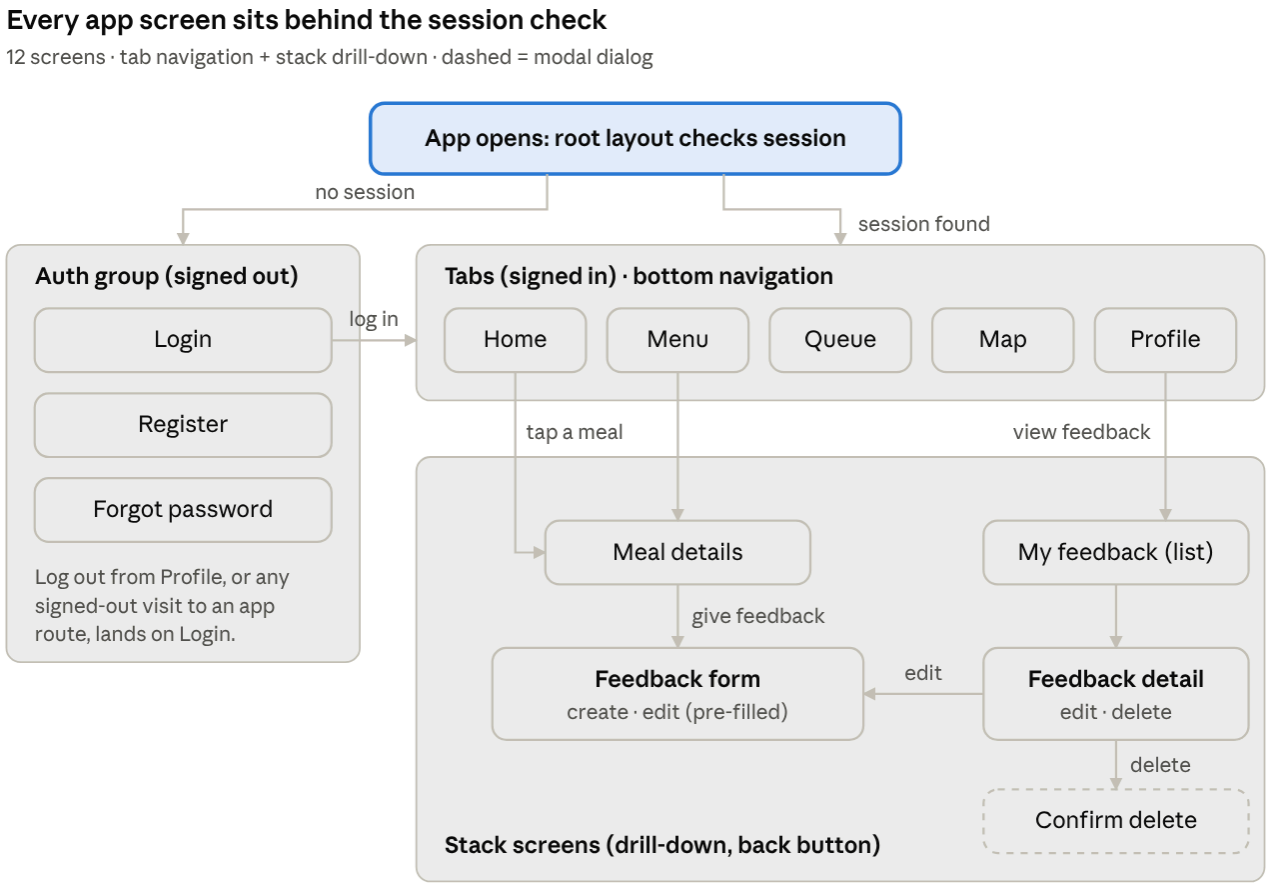
Table	Read	Insert	Update	Delete
profiles	Own row	Sign-up trigger only	Own row, `full_name` column only	None
meals	Signed-in users	Admin	Admin	Admin
queue_status	Signed-in users	Admin	Admin	Admin
mess_locations	Signed-in users	Admin	Admin	Admin

Table	Read	Insert	Update	Delete
feedback	Own rows	Own rows (`user_id` defaults to `auth.uid()`)	Own rows, `rating` and `comment` columns only	Own rows

"Admin" is `public.is_admin()`, a security-definer function reading `profiles.role`. Because students cannot update `role`, nobody can promote themselves. These rules were tested in Postgres with two student accounts: a student saw only their own feedback, could not edit or delete another student's review, could not forge `user_id`, could not change the menu, and could not make themselves admin.

4. Screen flow

The app has 12 screens and two navigation patterns: a bottom tab bar for the five main sections and a stack for drill-down screens. The `(app)` layout is the authentication boundary.



screen flow · 12 screens, auth boundary at the root layout

Home cards also open the Queue and Map tabs. Meal reminder settings are a section of the Profile screen. Because the redirect lives in `app/(app)/_layout.tsx`, a signed-out user cannot reach any app screen by changing the navigation state or opening a deep link.

Screen	Expo Router file	Reached from
Login	`app/(auth)/login.tsx`	App start with no session, Log out
Register	`app/(auth)/register.tsx`	Login
Forgot password	`app/(auth)/forgot-password.tsx`	Login
Home	`app/(app)/(tabs)/index.tsx`	Tab bar, after login
Menu	`app/(app)/(tabs)/menu.tsx`	Tab bar
Queue	`app/(app)/(tabs)/queue.tsx`	Tab bar, Home card
Map	`app/(app)/(tabs)/map.tsx`	Tab bar, Home card
Profile (incl. meal reminders)	`app/(app)/(tabs)/profile.tsx`	Tab bar
Meal details	`app/(app)/meal/[id].tsx`	Menu, Home
My feedback	`app/(app)/feedback/index.tsx`	Profile
Feedback detail	`app/(app)/feedback/[id].tsx`	My feedback, Meal details
Feedback form	`app/(app)/feedback/form.tsx` (`?mealId=` create, `?id=` edit)	Meal details, Feedback detail

5. API integration table

Two integrations sit beyond auth and the database, and both are load-bearing. The map's markers come from the `mess_locations` table and the device position drives the distance shown; reminders are scheduled from each meal's `serving_start` in the database and the student's saved choices.

Integration	Provider / package	Endpoints and functions used	What the data is used for	Rate limits	When the call fails
Maps & Location (Group A)	`expo-location`; `react-native-maps` (Google Maps SDK on Android)	`requestForegroundPermissionsAsync`, `getCurrentPositionAsync` (15 s timeout); `<MapView>`, `<Marker>`; Google Maps walking-directions URL via `Linking.openURL`	Student's latitude/longitude → Haversine distance and walking minutes to the mess; mess markers from `mess_locations`	Location runs on the device, no limit. Map loads on Android are free for the Maps SDK; the key is restricted to the app's package and signing certificate	Permission denied: map still shows the mess marker, with an explanation and an "Allow location in Settings" button. GPS timeout: message and "Retry location". Locations query fails: error state with Retry. No locations saved: empty state
Local notifications (Group B)	`expo-notifications`	`getPermissionsAsync`, `requestPermissionsAsync`, `setNotificationChannelAsync` (Android channel), `scheduleNotificationAsync` with a daily trigger, `cancelAllScheduledNotificationsAsync`	Reminder 15 minutes before each meal's `serving_start`; on/off choice per meal saved in AsyncStorage	On-device; Android caps an app at about 500 scheduled alarms, and the app uses at most 3	Permission denied: switches disabled, with an explanation and a link to Settings. Scheduling fails: error message, switch reverts, choice not saved. No menu posted yet: reminders use the usual serving times, and the screen says so

Integration	Provider / package	Endpoints and functions used	What the data is used for	Rate limits	When the call fails
Auth + database (not counted)	Supabase JS client v2	`auth.signUp`, `signInWithPassword`, `signOut`, `resetPasswordForEmail`, `verifyOtp`, `updateUser`, `onAuthStateChange`; PostgREST select / insert / update / delete	Session, profile, meals, queue reports, feedback	Auth emails rate-limited per address (one per ~60 s) and per hour	TanStack Query retries twice, then an error state with Retry; auth errors are mapped to plain messages ("Email or password is incorrect", "Too many attempts…")

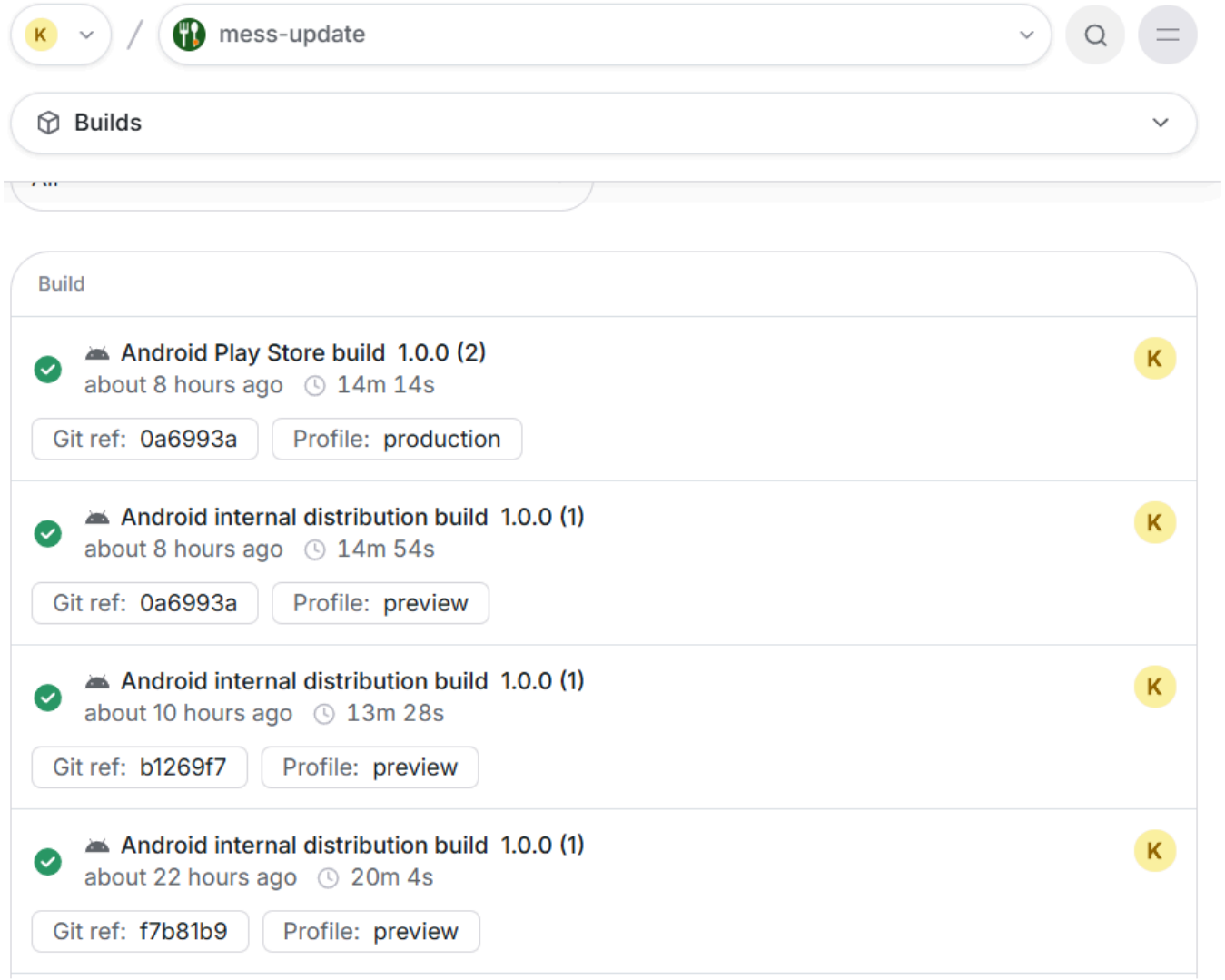
Every data source shows four states: **loading** (spinner with a label), **success**, **empty** (for example "No menu yet — the mess hasn't posted today's meals" or "No feedback yet") and **error** (plain message plus Retry). With no connection, the last loaded menu and queue stay visible from the AsyncStorage cache, and the app does not crash in aeroplane mode.

6. Build record

Both artefacts were built with EAS Build from the `preview` (APK) and `production` (AAB) profiles in `eas.json`, signed with an upload keystore that EAS generated and stores against the Expo account `@kennynauna`.

Artefact	Profile	EAS build ID	Result
APK (first attempt)	preview	086cc1c6-5ef7-475e-ba79-9962c269ffdd	Failed at Install dependencies (lock-file mismatch, fixed below)
APK (first success)	preview	d450d2d7-b097-48c4-94ee-665949fff493	Built; installed on a physical Android phone
APK (diagnostic)	preview	8fe439f4-d962-4f48-8208-891510ea10b1	Built; showed the real cause of the login failure (see below)
APK (submitted)	preview	58c49232-ae51-45a0-b8e7-6ed1228c559e	Finished in 14 min 54 s (Git ref 0a6993a)
AAB (submitted)	production	4abeeb27-808a-441b-91a5-11db80588dba	Finished in 14 min 14 s (Git ref 0a6993a); versionCode 2

EAS dashboard: both submitted builds finished successfully from the same commit, `0a6993a` (production AAB 1.0.0 (2) and preview APK 1.0.0 (1)), shown with the earlier preview builds.



Configuration changes needed for the compiled build

Change	Why
Upgraded Expo SDK 54 → 57	The Expo Go on the test phone was SDK 57; one SDK is used throughout
Supabase URL and publishable key stored as EAS environment variables (`preview`, `production`)	`.env` is git-ignored, so cloud builds cannot read it; `EXPO_PUBLIC_` values are inlined at build time

Change	Why
Pinned `react-native-reanimated` 4.5.1 and `react-native-worklets` 0.10.1 in `package.json`	Expo Router pulls these in as peer dependencies; the local npm 11 and EAS's npm 10 resolved different versions, so `npm ci` refused to install. Pinning the SDK-matched versions made the lock file identical everywhere
`app.config.ts` reads `GOOGLE_MAPS_ANDROID_API_KEY`, with a placeholder when it is missing	Google Maps on Android crashes the app if no key is declared; the placeholder prevents the crash. The real key is read from an EAS environment variable; without it the map background can render blank, but the mess marker and distance still work
Added the `expo-status-bar` plugin; `eas init` added `extra.eas.projectId` and `owner`	Required by the SDK 57 config and to link the project to EAS
Android package `com.kennynauna.messupdate`, custom icon, adaptive icon and splash, only location permissions declared	Required configuration; avoids the default Expo icon and over-requesting permissions
`appVersionSource: remote`, `autoIncrement` on production	EAS manages `versionCode` so each Play upload increases it

Problem found when testing the first APK. Login, registration and password reset all failed with a generic message. A diagnostic build that shows the underlying error revealed `UnknownHostException: Unable to resolve host …supabase.co`: the free Supabase project had been paused automatically after seven days without activity. Resuming it in the dashboard and re-running `seed.sql` for the current week fixed it with no code change. The project must stay active until marking is complete.

7. Reflection

What I built. Mess Update is a campus app that answers one question before a student walks to the mess: is it worth going now? It shows today's meals and whether they are being served, the latest queue level with a wait range and how old that report is, the walk to the mess on a map, and a private record of the meals I have rated. Students sign up, log

in, stay logged in and reset a forgotten password with an emailed code. Feedback has full create, read, update and delete, and the database, not the app, decides who can see and change each row. Reminders 15 minutes before each meal are scheduled on the phone.

What broke. The biggest lesson was that "it works in Expo Go" and "it works as an app" are different claims. My first EAS build failed at installing dependencies. Expo Router pulls in Reanimated and Worklets as hidden peer dependencies; my computer's npm resolved one version and the build server's npm resolved another, so `npm ci` refused to install. The fix was to install the SDK-matched versions explicitly so the lock file was the same everywhere. A second problem was environment variables: my `.env` file is correctly ignored by Git, which also means the cloud build cannot see it, so I had to store the Supabase URL and publishable key as EAS environment variables. I also learned that Google Maps on Android needs an API key in the compiled app, and without one the map can crash the app, so I added a safe placeholder and planned for the real key. When I installed the first APK, login, registration and password reset all failed with a generic error. I rebuilt the app to show the underlying error, which said the phone could not resolve my Supabase address: the free project had paused itself after a week without use. Resuming it fixed everything, and I learned to show a reference for unexpected errors instead of hiding them.

Smaller things broke along the way. Expo Go updated itself to a newer SDK than my project and stopped opening it. Supabase sent an 8-digit reset code while my form only accepted 6 digits. Gmail had to be set up as a custom SMTP sender before I could edit the email templates. Just before submission, running the linter found unescaped apostrophes in screen text and a state update started inside an effect, which I fixed. Each was a reminder to check the actual error message rather than guess.

What I cut and why. I deliberately cut in-app admin screens, meal photos, a weekly menu, student queue reports and reverse geocoding. Admins use the Supabase dashboard instead of an admin UI, which kept the scope to what one person could build, test and compile properly. Meal photos were a good idea that would have made a third integration, but with limited time I chose to finish the must-haves and the compiled build first. I also chose "reported" queue status over any claim of a live crowd count, because without sensors the app cannot honestly know how many people are in the line; showing the report's age is more truthful.

What I would do differently. I would set up GitHub with feature branches and small daily commits from the first week instead of committing in bursts, because my history has a long gap in the middle of the semester and my last few fixes went straight to main. I would produce the first APK in Week 8, as the brief recommends, rather than near the end, since most of my real problems only appeared in the compiled build. I would also pin the SDK

version and turn off Expo Go auto-updates on day one, and keep one copy of the project in one folder; having duplicate copies and a nested folder caused repeated "package.json not found" errors.

What I learned. Security belongs in the database. Writing Row Level Security policies and testing them with two accounts showed me that hiding a button in the app protects nothing, while a policy on the table protects everything. I learned to separate code into routes, services and hooks so screens never call the database directly, to validate forms with Zod, and to design every screen for loading, empty and error states. Most of all I learned the release side of mobile development: package identifiers, signing keys, environment variables at build time, and reading build logs line by line until the real error appears.

Use of AI. I used Claude to review my plan against the brief, scaffold the project structure, write the first versions of the SQL policies and screens, help diagnose build and lint errors, and draft this report from my notes and test results. I reviewed, ran and tested every part, and I can explain each file.